

# Phase 03 — HTTP and HTTPS (Complete Notes)

*The language spoken between browser and server — and how it's secured.*

## 1. What is HTTP?

**HyperText Transfer Protocol** — the application-layer (L7) protocol clients and servers use to exchange requests and responses. It rides on top of TCP (Phase 1). It is **text-based**: you can read raw HTTP with your eyes.

**Analogy:** TCP is the phone line; HTTP is the language spoken over it. DNS found the number, TCP dialed it, HTTP is the conversation.

## 2. The HTTP Request

```
POST /api/users HTTP/1.1      ← request line: METHOD path VERSION
Host: api.example.com         ← headers (key: value)
Content-Type: application/json
Authorization: Bearer eyJhbG...
Content-Length: 45

{"name": "Raihan", "role": "backend"} ← blank line separates headers from body
                                     ← body (optional)
```

### Methods (verbs)

| Method  | Meaning                               | Idempotent? | Body?  |
|---------|---------------------------------------|-------------|--------|
| GET     | read data                             | yes         | no     |
| POST    | create / do something                 | <b>no</b>   | yes    |
| PUT     | replace entirely                      | yes         | yes    |
| PATCH   | update partially                      | no*         | yes    |
| DELETE  | remove                                | yes         | rarely |
| OPTIONS | "what do you allow?" (CORS preflight) | yes         | no     |

**Idempotent** = calling it 5 times has the same effect as once. This matters in production: browsers/proxies may safely *retry* idempotent requests; retrying a POST can double-charge a customer.

### 3. The HTTP Response

```
HTTP/1.1 201 Created           ← status line
Content-Type: application/json
Set-Cookie: SESSIONID=abc123; HttpOnly; Secure
Cache-Control: no-store

{"id": 17, "name": "Raihan"}
```

#### Status codes — know these cold

```
2xx SUCCESS      200 OK    201 Created    204 No Content (e.g. after DELETE)
3xx REDIRECT     301 Moved Permanently  302 Found (temporary)  304 Not Modified (cache!)
4xx CLIENT'S FAULT 400 Bad Request  401 Unauthorized (who are you?)
                  403 Forbidden (I know you – not allowed)  404 Not Found
                  409 Conflict  422 Unprocessable  429 Too Many Requests (rate limit)
5xx SERVER'S FAULT 500 Internal Error  502 Bad Gateway (proxy: backend dead ★)
                  503 Unavailable  504 Gateway Timeout (proxy: backend too slow ★)
```

★ You will meet 502/504 constantly once NGINX (Phase 5) sits in front of your app: **502 = NGINX couldn't reach/got garbage from Spring Boot; 504 = Spring Boot too slow**. Golden rule: 4xx → blame the caller; 5xx → blame yourself.

#### Important headers

| Header               | Direction | Purpose                                                         |
|----------------------|-----------|-----------------------------------------------------------------|
| Host                 | →         | which site (one IP can host many domains — key to NGINX vhosts) |
| Content-Type         | ↔         | body format ( application/json )                                |
| Authorization        | →         | credentials ( Bearer <token> )                                  |
| User-Agent           | →         | client software                                                 |
| Cache-Control , ETag | ←         | caching rules                                                   |
| Set-Cookie / Cookie  | ←/→       | state (below)                                                   |
| X-Forwarded-For      | →         | original client IP, added by proxies (Phase 5!)                 |

### 4. Statelessness, Cookies, Sessions

**HTTP is stateless:** each request is independent; the server remembers nothing between requests by default. Statelessness is a *feature* — it's what lets you scale to many servers (Phase 9).

So how do logins work? Two patterns:

#### Cookies + server-side sessions

1. POST /login (credentials)
2. Server stores session {abc123 → user 17} in memory/Redis  
Response: Set-Cookie: SESSIONID=abc123; HttpOnly; Secure
3. Browser automatically sends Cookie: SESSIONID=abc123 on every later request
4. Server looks up abc123 → knows it's user 17

Cookie flags you must know: `HttpOnly` (JS can't read it → blocks XSS theft), `Secure` (HTTPS only), `SameSite` (CSRF defense), `Max-Age`.

## Tokens (JWT) — the stateless alternative

Server signs a token containing the user id; client sends it in `Authorization: Bearer ...`; server *verifies the signature* instead of looking anything up. No server-side storage → scales trivially; trade-off: hard to revoke before expiry. (Session storage in Redis returns in Phase 9.)

## 5. REST API communication

REST = using HTTP the way it was designed: **URLs are nouns, methods are verbs, status codes are the outcome.**

|        |                                |                                   |                       |
|--------|--------------------------------|-----------------------------------|-----------------------|
| GET    | /api/orders                    | list orders                       | 200                   |
| POST   | /api/orders                    | create order                      | 201 + Location header |
| GET    | /api/orders/42                 | one order                         | 200 / 404             |
| PUT    | /api/orders/42                 | replace order                     | 200                   |
| DELETE | /api/orders/42                 | delete order                      | 204                   |
| GET    | /api/orders?status=paid&page=2 | filtering/paging via query params |                       |

Anti-patterns you'll be asked about: verbs in URLs ( `/getOrders` , `/orders/42/delete` ), returning 200 with `{"error": "..."}`  in the body, using GET for state changes.

## 6. HTTPS — what problem does it solve?

Plain HTTP is readable by **everyone in the path**: WiFi neighbors, ISP, any router. Three attacks:

1. **Eavesdropping** — reading your passwords.
2. **Tampering** — modifying responses (injecting ads/malware).
3. **Impersonation** — a fake server pretending to be your bank.

HTTPS = HTTP over **TLS** (Transport Layer Security; SSL is its obsolete ancestor, the name stuck). It gives **encryption** (nobody reads), **integrity** (nobody modifies), **authentication** (server proves identity).

## 7. The crypto building blocks

### Symmetric encryption

One shared key locks and unlocks (AES). Very fast. Problem: how do two strangers agree on the key over a wire everyone can read?

### Asymmetric encryption (public/private key)

A key **pair**: what the public key encrypts, only the private key decrypts. Public key = an open padlock you hand out freely; private key = the only key that opens it. Anyone can lock a box for you; only you open it. Slow — so it's used only to *bootstrap* a symmetric key, then AES does the bulk work.

### Certificates & Certificate Authorities

How do you know the public key really belongs to `example.com` and not an attacker? A **certificate**: the server's public key + domain name, **digitally signed by a Certificate Authority (CA)** (Let's Encrypt, DigiCert...). Your OS/browser ships with a trusted list of CA root certificates. Signature chain: your cert ← intermediate CA ← trusted root.

**Analogy:** a passport. Anyone can claim a name; the passport binds name to photo and is issued by a government your border control already trusts.

## 8. The TLS handshake — exactly what happens

(after TCP handshake, before any HTTP)

```
Client → ClientHello: TLS versions + ciphers I support, random1
Server → ServerHello: chosen cipher, random2
Server → CERTIFICATE (public key + domain, CA-signed)
Client:  verify – CA signature trusted? domain matches? not expired?
Client+Server → key exchange (ECDHE) → both derive the same SESSION KEY
              (the private key never travels; eavesdroppers can't derive the key)
Both → "Finished" – everything after this is AES-encrypted
      — now normal HTTP flows inside the encrypted tunnel —
```

- Modern TLS 1.3 does this in **1 round trip** (~1 handshake added latency).
- **ECDHE** gives *forward secrecy*: session keys are ephemeral, so even a stolen private key can't decrypt yesterday's recorded traffic.
- What an observer still sees: destination IP and (usually) domain name (SNI) — but no paths, headers, cookies, or bodies.

### Full journey so far

```
type https://example.com
→ DNS (name→IP) → TCP handshake → TLS handshake → HTTP request/response (encrypted)
```

## 9. Commands & labs

```
curl -v https://example.com          # watch DNS, TCP connect, TLS (certificate lines), then > < HTTP
curl -i https://api.github.com/users/octocat      # -i: show response headers
curl -X POST -H "Content-Type: application/json" \
  -d '{"title":"test"}' https://jsonplaceholder.typicode.com/posts
curl -I https://google.com            # HEAD: headers only – spot the 301 redirect
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com # status + timing

# inspect a real certificate: issuer (CA), validity dates, domain (CN/SAN)
openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/null | openssl
x509 -noout -issuer -subject -dates

# raw HTTP by hand – type it yourself against a local server!
python3 -m http.server 8000          # terminal 1
nc localhost 8000                    # terminal 2, then type EXACTLY:
GET / HTTP/1.1
Host: localhost

# (press Enter twice) → raw response appears
```

Also: open browser DevTools (F12) → Network tab → reload any site → click a request → inspect headers, status, cookies, timing. Live there.

## 10. Common mistakes

- Returning 200 for errors, or 500 for bad user input (should be 4xx).

- POST for reads / GET for writes; retry logic on non-idempotent endpoints.
- Cookies without `HttpOnly` / `Secure` ; tokens in `localStorage` when cookies+flags fit better.
- "We have HTTPS so we're secure" — TLS protects the *transport* only; SQL injection, weak auth, IDOR are untouched.
- Serving the site on HTTPS but forgetting the 80→443 redirect (Phase 5 fixes this in NGINX).
- Letting certificates expire (browsers hard-block). Automate renewal (certbot — Phase 5/7).

## 11. Interview questions

---

1. What happens when you type `https://example.com` ? (DNS → TCP → TLS → HTTP — with detail at each step.)
2. GET vs POST vs PUT vs PATCH; what does idempotent mean and why does it matter for retries?
3. 401 vs 403? 502 vs 504 — which layer produces them?
4. HTTP is stateless — what does that mean, and how do sessions exist anyway? Cookie-session vs JWT trade-offs?
5. Walk through the TLS handshake. Why use asymmetric crypto only at the start?
6. What is a CA? What exactly does the browser verify in a certificate?
7. What can a network observer still see when you use HTTPS?

## 12. ASSIGNMENT 03 (submit to Claude)

---

1. Run the `nc` raw-HTTP lab and paste your hand-typed request + the response. Label every part (request line, headers, status line, body).
2. Design REST endpoints for a food-delivery app: restaurants, menus, orders, order status updates. Table: method, path, success code, failure codes.
3. `curl -v https://github.com 2>&1 | head -40` — annotate: which lines are TCP, which TLS, which HTTP?
4. From memory: write the TLS handshake story as a dialogue between Browser and Server (like a movie script).
5. Your API returns 502 sometimes under load, 504 other times. What is the difference in root cause? Which component generates each?
6. Explain to a junior why "just retry all failed requests 3 times" is a dangerous policy for a payments API. Which HTTP concept is at play?